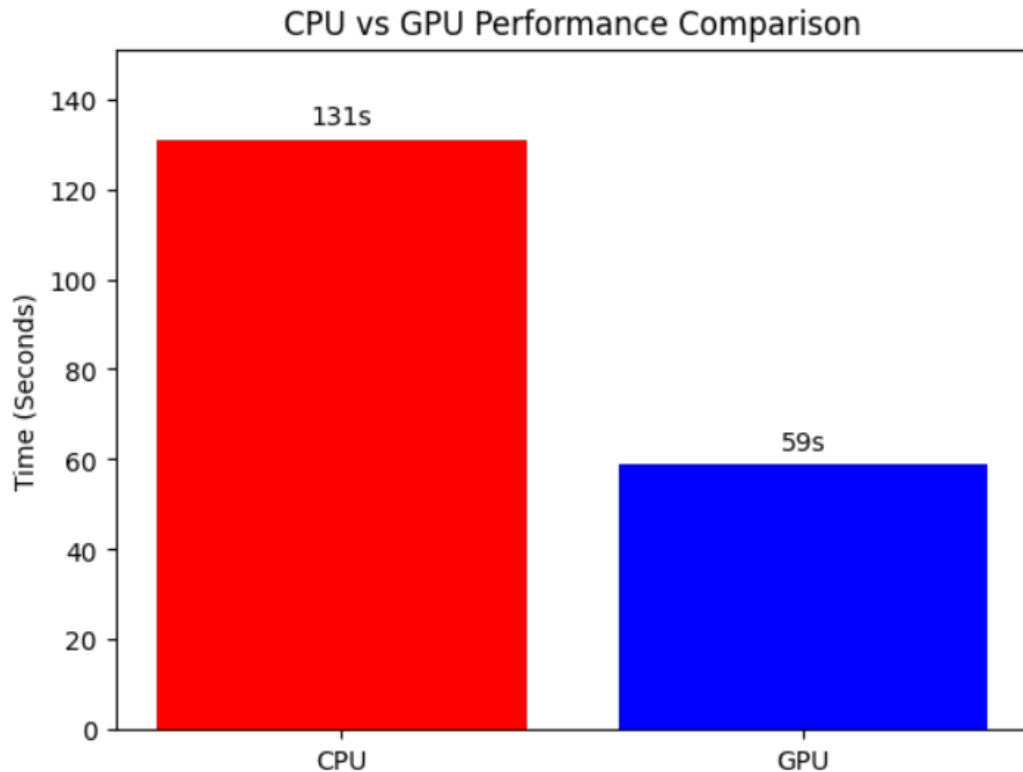


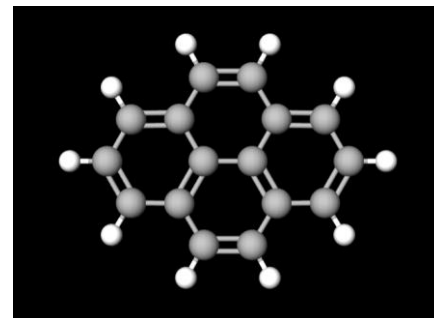
Work Done by TEJJAN ARORA

TEAM SAKEC

CPU vs GPU Comparison Plot (Pyrene Molecule)



These are
Pure DFT runs.



Pyrene ($C_{16}H_{10}$)

This comparison is justified by the fact that the GPU and CPU calculations produced identical energies (i.e. -615.760309 Hartree).

CPU-GPU Interoperability Hurdles

Problem: 1) Core libraries (**PySCF/Qiskit**) rely on **C/Rust** compiled backends that expect pointers to pinned CPU memory.

2) **CuPy** store data in **VRAM**, which these backends cannot directly access, triggering standard Python safety exceptions before memory corruption occurs :

- **PySCF:** CuPy arrays trigger validation errors (e.g., **AssertionError** during density matrix operations).
- ~~**Qiskit:** CuPy blocks automatic device-to-host conversions, causing a **TypeError**.~~

Diagnostic tool used : Dynamic Namespace Injection

- Used Python's **sys.modules** to intercept all **NumPy imports**.
- Redirected them to the **CuPy GPU library** during runtime initialization.
- ~~This forced the program to use GPU arrays instead of CPU arrays without modifying the original source code.~~

API Differences Between NumPy and CuPy

- **Missing administrative modules:** Some NumPy utilities (e.g., `numpy.lib.NumpyVersion`) are not implemented in CuPy.
- **Memory immutability:** Qiskit uses NumPy's `.setflags(write=False)` to protect quantum gate definitions, which CuPy does not support.
- **Type conversion behavior:**
 - NumPy automatically converts **Python lists/tuples** → **arrays** during matrix operations.
 - ~~CuPy requires **explicit conversion**, otherwise it raises errors.~~

Monkey Patching for Backend Boundary Mapping

Diagnostic tool : Runtime Attribute Mocking

1) Bypassing Version Checks

- Created a **mock version of `numpy.lib.NumpyVersion`**.
- This allowed **Qiskit initialization** to proceed even though NumPy was replaced with CuPy.

1) Bypassing Memory Locks

- Added a **dummy `.setflags()` method** to the `cupy.ndarray` class.
- This bypassed Qiskit's **read-only memory protection** used for quantum gate definitions.

Hardware Latency (PCIe Bottleneck)

- Data must move between **CPU RAM and GPU VRAM** through the **PCIe bus**.
- For **small systems** (e.g., active space: **6 orbitals, 2 electrons**), **data transfer time exceeds the GPU compute benefit**.

Current Error : Execution stops during **quantum gate construction** with the error:

AttributeError: 'list' object has no attribute 'repeat'

Root Cause

- **Qiskit depends on NumPy's loose typing behavior**.
- NumPy automatically converts **Python lists** → **NumPy arrays** during matrix operations.
- **CuPy does not allow this implicit conversion**, because it would require **hidden CPU → GPU memory transfers**.

Assumptions for more patching

Limitations of Further Patching

- Fixing this single function using **monkey patching** would only expose **more API mismatches** later.
- Similar issues would appear in functions such as: `stack`, `concatenate` and other NumPy-dependent operations.

The Fundamental Barrier

- Even if **all Python-level API differences were patched**, the GPU arrays would eventually reach **PySCF's libcint core**.
- This compiled backend **requires CPU memory (NumPy arrays)**.
- Passing **GPU VRAM pointers** would cause a **fatal C-level memory access error**.

Assumption: Final Barriers to End-to-End GPU Execution

- **Circuit Initialization Failure:** As we have seen, when NumPy is replaced with CuPy, strict typing causes gate construction to crash (e.g., `AttributeError: 'list' object has no attribute 'repeat'`).
- **Rust Backend Constraint:** Even if GPU chemistry runs via **gpu4pyscf**, the results must pass through Qiskit Nature (e.g., **ActiveSpaceTransformer, ParityMapper**), whose core Rust backend requires contiguous CPU NumPy memory.
- **Outcome:** Passing GPU arrays leads to **TypeError (implicit conversion blocked)** or **segmentation faults**, preventing full GPU execution.

Code modifications

```
import sys
import cupy as cp

# Fake the numpy.lib.NumpyVersion class so Qiskit initializes without crashing
class DummyNumpyVersion:
    def __init__(self, *args, **kwargs): pass
    def __lt__(self, other): return False
    def __ge__(self, other): return True

class DummyLib:
    NumpyVersion = DummyNumpyVersion

cp.lib = DummyLib()
cp.__version__ = '1.23.5' # Fake the NumPy version

# Fake the setflags method on cupy arrays to bypass Qiskit's gate locking
def dummy_setflags(self, *args, **kwargs):
    pass # Do absolutely nothing

cp.ndarray.setflags = dummy_setflags # Monkey-patch the method into CuPy

sys.modules['numpy'] = cp



---


mol = pyscf.M(atom=driver._kwargs.get('atom', driver.atom), basis=driver._kwargs.get('basis', driver.basis))

# Initialize GPU-accelerated RKS
mf = gpu4pyscf.dft.rks.RKS(mol)
mf.xc = driver._kwargs.get('xc_functional', driver.xc_functional)
mf.kernel() # Run the GPU calculation

# Inject the GPU results back into the Qiskit driver so it can parse the MOs
driver._calc = mf
driver._mol = mol
```

CPU vs. GPU Task Delegation

CPU

Optimal For: Sequential logic, branching, and complex decision-making.

The SciPy Optimizer (L-BFGS-B): Must look at Step N before deciding the direction for Step N+1. Strictly serial.

The Embedding Controller: Managing the while loop and calculating the DIIS matrix damping.

Dynamic Circuit Routing: Python interpreting the logic of where to place quantum gates.

CPUs have a few incredibly fast, highly intelligent cores. Putting serial tasks on a GPU leaves **99.9% of the GPU idle**.

GPU

Optimal For: Massive, simultaneous floating-point math (SIMD).

Classical DFT (gpu4pyscf): Simultaneously calculating millions of electron-electron repulsion integrals.

Quantum Statevector: Holding 2^{12} or 2^{20} complex numbers in memory and updating them all at once.

Adjoint Gradients: Calculating the exact slope for all parameters simultaneously via tensor contraction.

The A100 executes massive math matrices in a fraction of a millisecond.

Aer Estimator

Consistently attempted to integrate AerEstimator(device="GPU") but face stability roadblocks.

The Python Parameter Bottleneck : In every iteration of a SciPy optimization loop, Qiskit must "bind" new parameters to the **3,500-gate UCCSD circuit**. Because this overhead is handled in Python, it creates a massive lag before the circuit even reaches the simulator.

PCIe Bus Starvation: Even with `device="GPU"`, AerEstimator functions as a decoupled C++ backend. The CPU must package the circuit, ship it across the **PCIe bus** to the A100, and wait for the result to ship back.

Lack of Native Adjoint Differentiation: AerEstimator typically treats the circuit as a "black box," relying on **Parameter Shift** or **Finite Differences** to find gradients. This requires evaluating the circuit hundreds of times (once for every parameter shift).

Result: Forced to revert back to simple noiseless **CPU Estimator** for stability and speed.

GPU4PySCFDriver Class

- **The Standalone Script:** Standard `gpu4pyscf` is great for getting a quick total energy, but it strands the chemistry matrices (CuPy arrays) on the GPU.
- **The Hijack:** Created a custom Python class that inherits all the normal rules of Qiskit's PySCFDriver, but we overwrote the `run_pyscf()` method.
- **The GPU Offload:** Instead of calling standard CPU PySCF, we force it to call `gpu_dft.RKS(mol)`. This pushes the massive initial Density Functional Theory (DFT) matrix calculations directly to the A100 GPU, finishing in seconds instead of minutes.
- **The Hand-off:** Qiskit expects CPU data. If we handed Qiskit a GPU array, it would crash. So, the final line of that class (`self._calc = gpu_calc.to_cpu()`) safely pulls the finished matrices back to the CPU so Qiskit can do the Active Space Embedding math.

The heaviest math of the initial Hartree-Fock simulation is completed in seconds on the GPU rather than minutes on CPU.

GPU4PySCFDriver Class

```
class GPUPySCFDriver(PySCFDriver):
    """Intercepts Qiskit's PySCF call to run the initial SCF on the GPU."""
    def run_pyscf(self):
        from pyscf import gto
        print(">> Building PySCF molecule...")
        atom_str = getattr(self, "atom", getattr(self, "_atom", None))
        basis_str = getattr(self, "basis", getattr(self, "_basis", None))
        chg = getattr(self, "charge", getattr(self, "_charge", 0))
        spn = getattr(self, "spin", getattr(self, "_spin", 0))

        self._mol = gto.M(atom=atom_str, basis=basis_str, charge=chg, spin=spn)
        self._mol.build()

        print(f">> Intercepting Qiskit Nature: Running gpu4pyscf on GPU...")

        # Using RKS for closed-shell Water (H2O)
        gpu_calc = gpu_dft.RKS(self._mol)
        gpu_calc.xc = self.xc_functional
        gpu_calc.grids.level = 3
        gpu_calc.run()

        print(">> Transferring GPU matrices back to CPU for Qiskit Mapper...\n")
        # This prevents Qiskit's ActiveSpaceTransformer from crashing on CuPy arrays
        self._calc = gpu_calc.to_cpu()
```

The "NumPy Trap" & Mentor Pivot

Mentor Suggestion: Pivot to SciPy L-BFGS-B for its mathematical robustness in chemistry optimization.

But, SciPy is built on NumPy, which is strictly CPU-bound. This forces a data round-trip for every iteration.

With just GPU4PySCF class, our VQE loop logic is stranded on the CPU while our initial integrals live on the GPU.

Qiskit Nature neither has an alternative for GPU portability, nor I was able to find any partial execution of the same on GitHub or research journals.

PennyLane & Adam Optimizer

The Commute Overhead

Attempted to use PennyLane for its reputed GPU acceleration and the Adam Optimizer.

Observation: Slower than a standard CPU-only run.

Cause: Every gradient step required thousands of tiny requests across the PCIe bus.

Outcome: Abandoned the Adam-PennyLane route to reduce communication frequency.

Data Transfer Penalties

When the "Quantum" part of the circuit is small (6 qubits), the overhead of copying parameters to VRAM outweighs the math speedup of the GPU cores.

Offloading is only efficient when the task size crosses the **Compute-to-Transfer Ratio** threshold.

Architectural Evolution Roadmap

Version 1

Framework bridge between
Qiskit Nature and PennyLane.
Limited by serial evaluation
and finite-difference noise.

Version 2

Transition to native Jordan-
Wigner mapping and Adjoint
Differentiation. Direct A100
GPU kernel execution.

Version 3

JAX-JIT compilation to
eliminate Python interpreter
bottlenecks for heavy circuits
/ molecules.

Version 1

Ansatz: Built by Qiskit Nature (`QiskitUCCSD`).

Mapper: `ParityMapper` (Deletes 2 qubits to save memory).

Execution: We had to use `bound_circuit.decompose()` and `qml.from_qiskit()` inside the loop to translate the Qiskit object into a PennyLane object.

Gradient: Relied on SciPy's default numerical Finite Differences (150+ evaluations per step).

Result: Worked, but took **9 hours** due to translation overhead and PCIe transfer starvation.

Version 2

Ansatz: Built natively by PennyLane (`qml.UCCSD`).

Mapper: Forced to switch to `JordanWignerMapper` because PennyLane's native templates strictly require 1 spatial orbital = 2 qubits.

Execution: Executed directly on `lightning.gpu`. No translation needed.

Gradient: We added `diff_method="adjoint"`.

We explicitly passed `gradient_fn = qml.grad(cost_fn)` to SciPy so it stopped guessing and used the exact analytical math.

Result: Solved Pyrene in **50 mins**. But starved on Tetracene because of PCIe bus.

Version 3

Addition: Imported `jax` and `jax.numpy`.

Decorators: Wrapped `@jax.jit` around the PennyLane `cost_fn` and the `gradient_fn`.

The Bridge: Added `scipy_cost` and `scipy_grad` helper functions to translate SciPy's standard numpy arrays into `jax.numpy` tensors.

Result: When ran Tetracene on Version 3, expected it to finish in minutes. Instead, it took 7 hours, and our GPU memory usage skyrocketed to a staggering 62 Gigabytes.

We discovered a fundamental limit of applying JIT compilation to Hybrid Quantum Embedding. In our embedding loop, the classical CPU calculates a new environment every iteration. Because the Hamiltonian physically changed, JAX was forced to throw away the 62 GB graph and re-compile the massive 3,500-gate circuit from scratch every iteration.

System Performance Comparison

Metric	V1 Architecture	V2 Native GPU
Execution Time (Pyrene)	~9.2 Hours	~52 Minutes
Gradient Method	Finite Differences	Adjoint (Exact)
GPU Utilization	0% – 5% (Stalled)	25% – 78% (Active)
Circuit Overhead	High	Moderate

System Performance Comparison

Metric	V2 Native GPU	V3 JAX-JIT
Execution Time (Tetracene)	~6.1 Hours	~7.1 Hours
Gradient Method	Adjoint (Exact)	Adjoint (Compiled C++)
GPU Utilization	Mostly 0% (Python Starved)	Mostly 0% (Compiler Starved)
Circuit Overhead	High	Extreme
Memory Footprint	~531 MB	~62 GB (XLA Graph Spike)

System Performance Comparison

Architecture Version	Final Total Energy (Hartree)	Total Runtime	Iterations to Converge	GPU Duty Cycle
Baseline (Pure Qiskit CPU)	-693.35251927 Ha	1648m 16s (~27.4 Hours)	36 Embedding Loops	N/A (CPU Only)
Version 2 (Native GPU)	-693.18845661 Ha	371m 19s (~6.1 Hours)	27 Embedding Loops	Severe Starvation (Python Overhead)
Version 3 (JAX-JIT GPU)	-693.18845499 Ha	423m 57s (~7.1 Hours)	29 Embedding Loops	Severe Starvation (Compilation Overhead)

System Performance Comparison

Performance Metric	Run 1: Qiskit Adjoint CPU	Run 2: PennyLane CPU	Run 3: PennyLane A100 GPU
Execution Hardware	Intel Xeon CPU	Intel Xeon CPU	NVIDIA A100 Tensor Core
Framework Base	Native Qiskit Nature	PennyLane lightning.qubit	PennyLane lightning.gpu
UCCSD Parameters	117	117	117
Total Embedding Iterations	36 Iterations	26 Iterations	26 Iterations
Total Wall-Clock Time	~5.23 Hours (313m 57s)	~6.06 Hours (363m 57s)	~6.14 Hours (368m 43s)
Final Computed Energy	-693.34300886 Ha	-693.18847956 Ha	-693.18844498 Ha

GraceHopper Chip

The **NVIDIA GH200** is a "superchip" that puts an ARM-based CPU (Grace) and an H100 GPU (Hopper) on the exact same physical package, connected by a proprietary bridge called NVLink-C2C.

The CPU and GPU can read the exact same memory addresses. They share Unified Memory. The CPU doesn't have to "send" the statevector to the GPU at all; the GPU just reaches over and reads it.

Result: The communication latency that crippled your A100 run would completely vanish. The GH200 would absolutely beat the standard Intel CPU on this specific run.

Compute Starvation: Even though the GH200 removes the communication bottleneck, your 12-qubit simulation is still fundamentally too small for a GPU.

THANK YOU